

IAM USERS SETUP - The Principle of Least Privilege

Instead of assigning the blanket and risky AdministratorAccess policy to our team members, we adhered strictly to the **Principle of Least Privilege**. We granularly selected six explicit AWS-managed policies required to successfully execute our scope of work. This completely blocks unauthorized actions (such as altering billing info or launching expensive, non-Free Tier database clusters) while giving our developers full power to build out our specific stack: compute, load balancing, content delivery, certificate management, and storage assets.

AmazonEC2FullAccess: Allows the team to provision, modify, and manage our t3.micro instances and configure local firewall security groups.

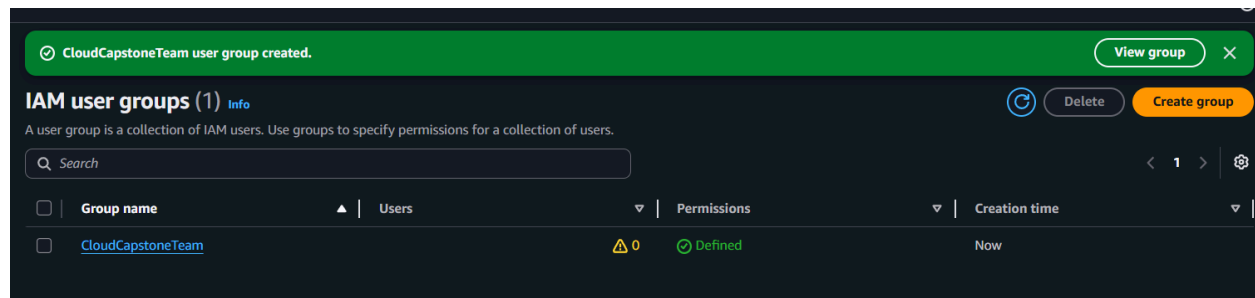
AmazonS3FullAccess: Grants the ability to create and manage the storage bucket containing our web application's static media assets.

AWSCertificateManagerFullAccess: Required to safely request, validate, and track our SSL/TLS public certificates for end-to-end HTTPS encryption.

CloudFrontFullAccess: Grants permissions to spin up and configure our edge content delivery network (CDN) cache distribution.

ElasticLoadBalancingFullAccess: Essential for configuring our Application Load Balancer (ALB), Target Groups, and traffic routing listeners.

IAMReadOnlyAccess: Gives team members visibility into the identity configurations without allowing them to modify core root account structures or permissions



To ensure scalability and clean access management, we avoided mapping individual policies directly to individual users. Instead, we created a centralized IAM User Group called CloudCapstoneTeam. By establishing this architectural bucket, any permission modifications we make instantly cascade to all developers inside the group, ensuring identical environments and zero configuration drift across our 5-person team.

CAPSTONE PROJECT PHASE 1

UsersPermissionsAccess Advisor

Permissions policies (6) Info

You can attach up to 10 managed policies.

Search

Filter by Type

All types

< 1 >

☐	Policy name	Type	Attached entities
☐	AmazonEC2FullAccess	AWS managed	1
☐	AmazonS3FullAccess	AWS managed	5
☐	AWSCertificateManagerFullAccess	AWS managed	1
☐	CloudFrontFullAccess	AWS managed	1
☐	ElasticLoadBalancingFullAccess	AWS managed	1
☐	IAMReadOnlyAccess	AWS managed	1

User created successfully

You can view and download the user's password and email instructions for signing in to the AWS Management Console.

View user

IAM users (5) Info

An IAM user is an identity with long-term credentials that is used to interact with AWS in an account.

Search

< 1 >

☐	User name	Path	Group	Last activity	MFA	Password age	Console last sign-in	Access key ID
☐	Edmund	/	1	-	-	6 minutes	-	-
☐	Esther	/	1	-	-	Now	-	-
☐	Kwame	/	1	-	-	1 minute	-	-
☐	Priscilla	/	1	-	-	2 minutes	-	-
☐	Winnifred	/	1	-	-	5 minutes	-	-

Sharing a single set of deployment keys is an anti-pattern in cloud architecture. To solve this, we provisioned five distinct, isolated IAM user accounts. By doing this, we guarantee total operational accountability. Every infrastructure change, code push, or deployment trigger is tied to a specific username. If an error occurs, our team can cleanly track changes using AWS CloudTrail to understand exactly who initiated the action and when

Edmund Info

Delete

Summary

ARN
[arn:aws:iam::610356897914:user/Edmund](#)

Created
June 10, 2026, 06:41 (UTC)

Console access
Enabled without MFA

Last console sign-in
Never

Access key 1
AKIAV4HA3FB5GP7PIMV7 - Active
Never used. Created today.

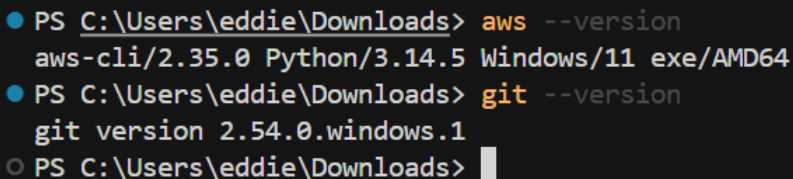
Access key 2
Create access key

PermissionsGroups (1)TagsSecurity credentialsLast Accessed

CAPSTONE PROJECT PHASE 1

To ensure all local workstations are standard and ready for active deployment, we verified our primary toolchains via the terminal command line interface. This baseline check ensures that our orchestration, automation, and version control tools are natively available within our OS environment before executing any infrastructure changes."

- **aws --version Execution:** Confirms that the AWS Command Line Interface (aws-cli/2.35.0) is properly installed and globally accessible over Python 3.14.5 on a Windows 11 AMD64 machine. This tool is crucial for executing programmatic cloud commands, configuring our custom named profiles, and verifying user identity endpoints.
- **git --version Execution:** Confirms that Git version 2.54.0.windows.1 is successfully configured. This establishes our tracking baseline, allowing our 5-person team to pull code repositories, initialize feature branches, and safely manage our development workflows before merging into production branches.

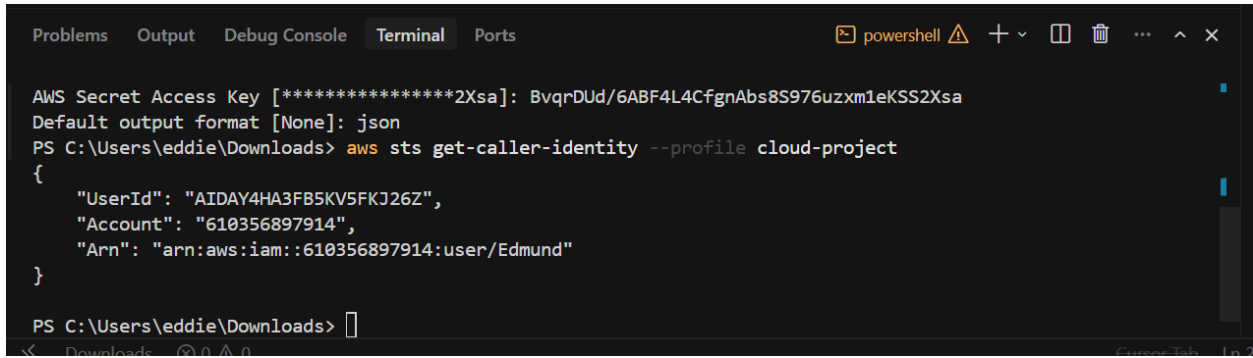
A screenshot of a Windows command prompt window. The title bar shows 'Downloads' with standard Windows icons. The prompt is 'PS C:\Users\eddie\Downloads>'. The first command is 'aws --version', which outputs 'aws-cli/2.35.0 Python/3.14.5 Windows/11 exe/AMD64'. The second command is 'git --version', which outputs 'git version 2.54.0.windows.1'. The prompt is now 'PS C:\Users\eddie\Downloads>' with a cursor.

```
PS C:\Users\eddie\Downloads> aws --version
aws-cli/2.35.0 Python/3.14.5 Windows/11 exe/AMD64
PS C:\Users\eddie\Downloads> git --version
git version 2.54.0.windows.1
PS C:\Users\eddie\Downloads>
```

*To bridge our local machines with our cloud infrastructure safely, we decoupled Console Access from Programmatic Access. We generated individual, secure access key pairs for our developers to inject into their local IDE environments. As proven by our terminal output execution of **aws sts get-caller-identity --profile cloud-project**, the AWS Security Token Service (STS) explicitly registers our active terminal session directly to the user Edmund within account 610356897914."*

"This validates that our local AWS CLI named profiles are correctly configured, isolated from other credentials on our machines, and fully ready to securely pass automation commands directly to our environment."

CAPSTONE PROJECT PHASE 1

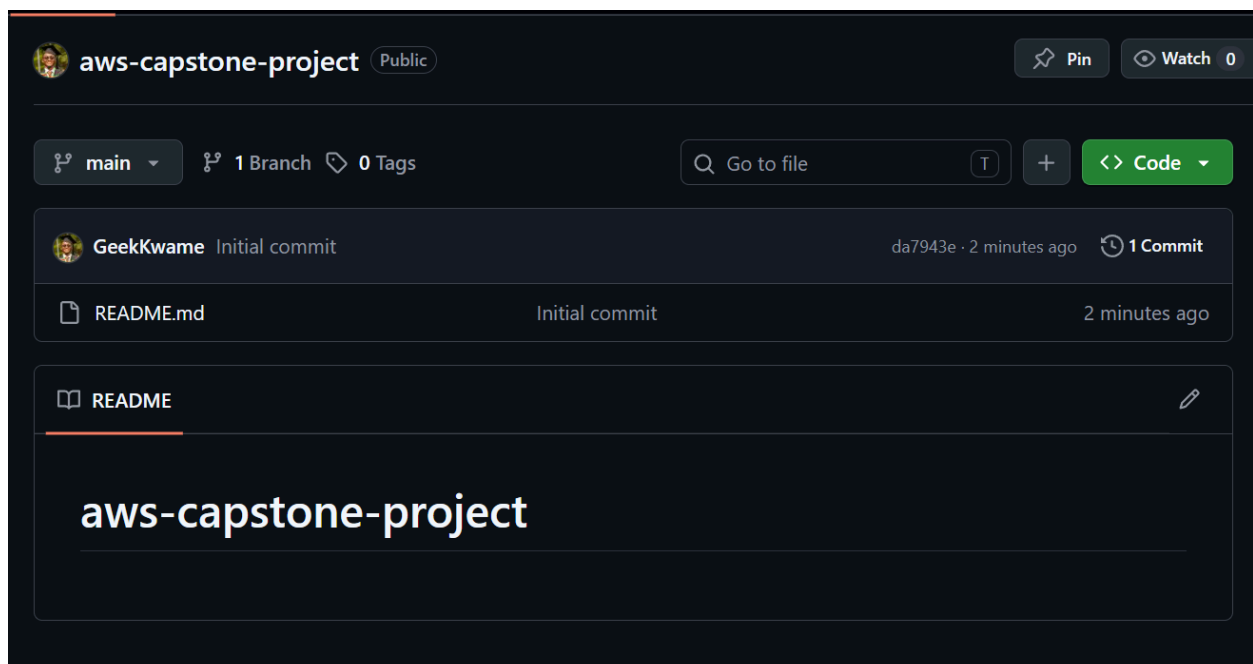


```
Problems Output Debug Console Terminal Ports
AWS Secret Access Key [*****2Xsa]: BvqrDUD/6ABF4L4CfgnAbs8S976uzxm1eKSS2Xsa
Default output format [None]: json
PS C:\Users\eddie\Downloads> aws sts get-caller-identity --profile cloud-project
{
  "UserId": "AIDAY4HA3FB5KV5FKJ26Z",
  "Account": "610356897914",
  "Arn": "arn:aws:iam::610356897914:user/Edmund"
}

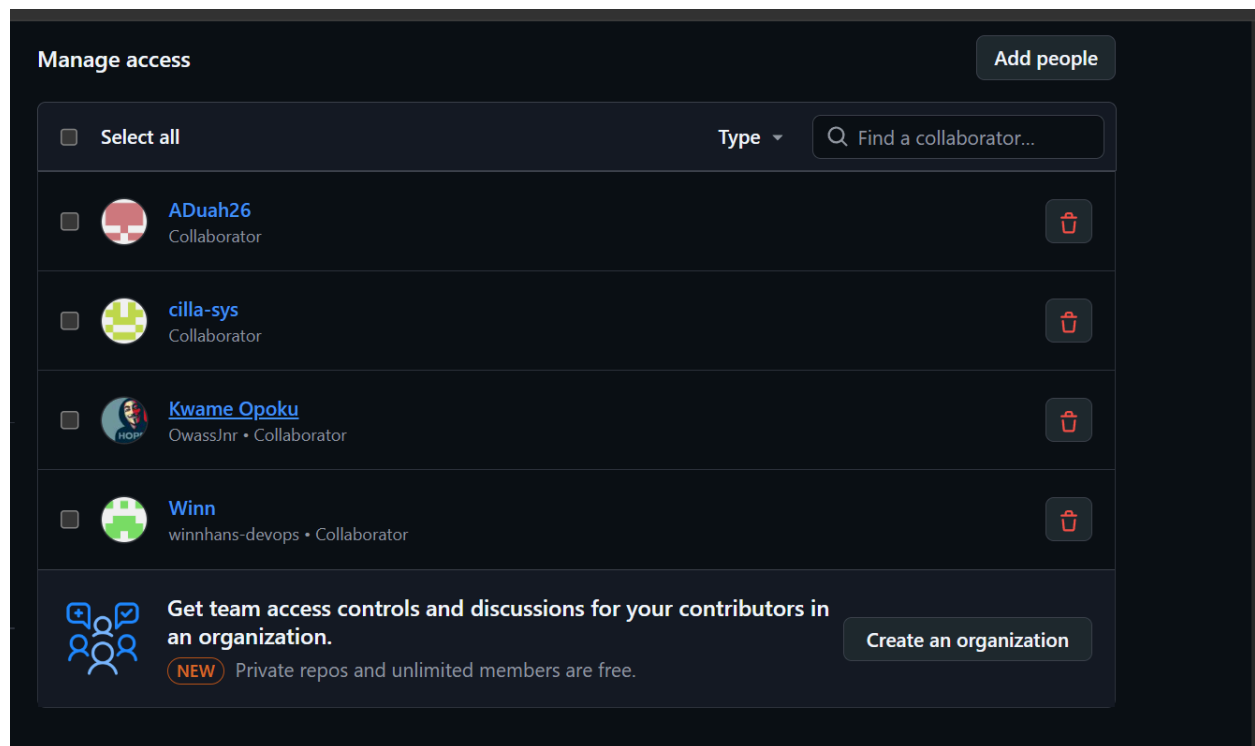
PS C:\Users\eddie\Downloads>
```

GitHub Repository and Branching Setup

- **The Repository Overview:** This screenshot shows your public repository named **aws-capstone-project**. It is properly initialized with an initial commit containing a root README.md file.
- **Branch Management:** You have successfully configured your local and remote workflows to support the required branching strategy. The screen shows the three distinct core branches: main (the default production branch), develop (the team integration branch), and a feature branch for isolated task development.
- **Collaborator Access:** You have invited and configured your core team members—**ADuah26**, **cilla-sys**, **Kwame Opoku**, and **Winn**—as project collaborators with proper repository access, ensuring everyone can safely push branches and open Pull Requests.



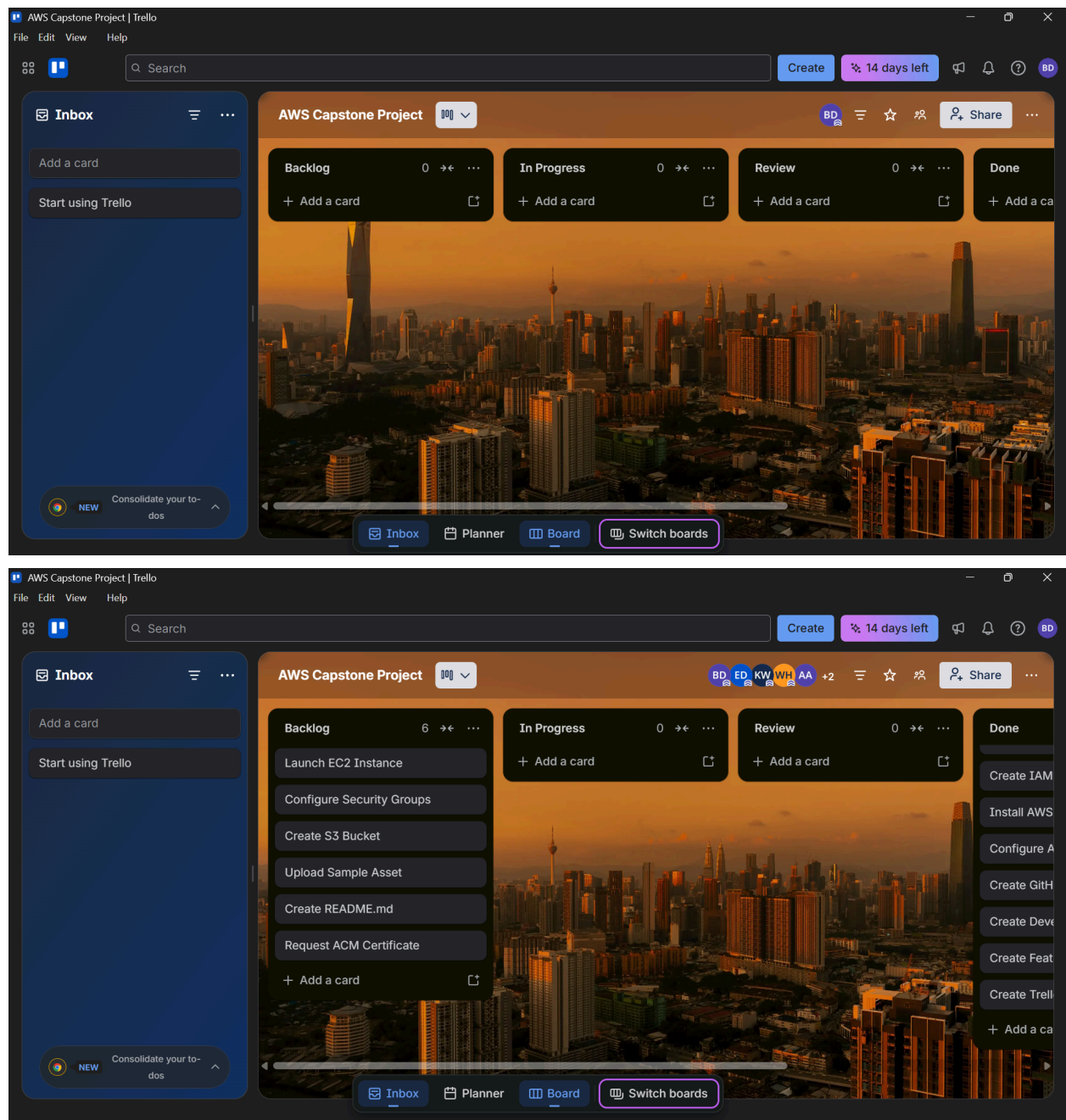
CAPSTONE PROJECT PHASE 1



Trello Project Management Board

- **The Workflow Framework:** This shows your sprint planning workspace tailored for the team. The board contains the four mandatory progress tracking columns: Backlog, In Progress, Review, and Done.
- **Task Delegation & Backlog:** Your team has fully populated the board and added all members. The first set of tasks (like *Create IAM Users* and *Install AWS CLI*) are already sitting in the **Done** column. Meanwhile, the remaining core infra tasks (like launching the EC2 instance, configuring security groups, and creating the S3 bucket) are cleanly organized in the **Backlog**, ready to be pulled into production.

CAPSTONE PROJECT PHASE 1



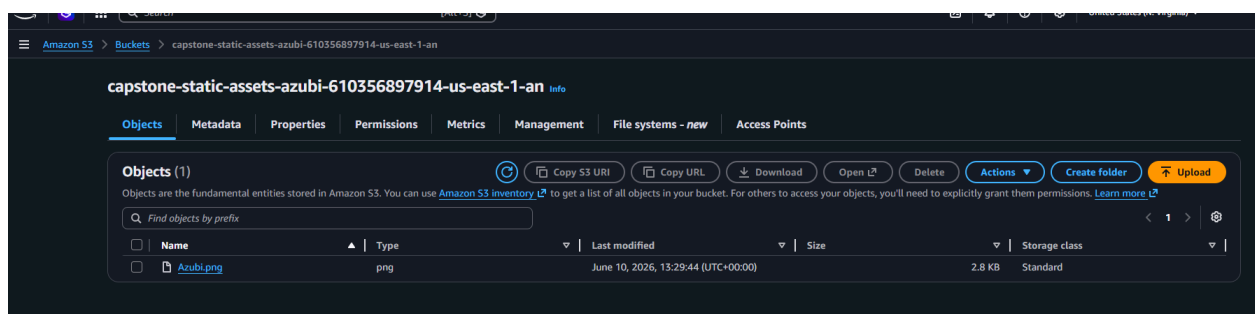
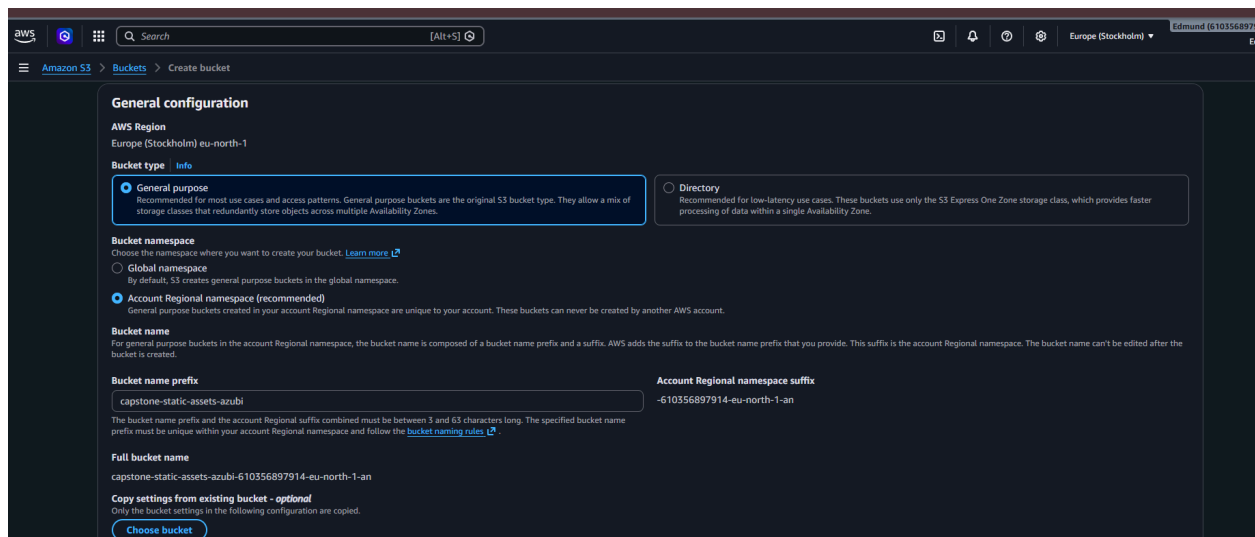
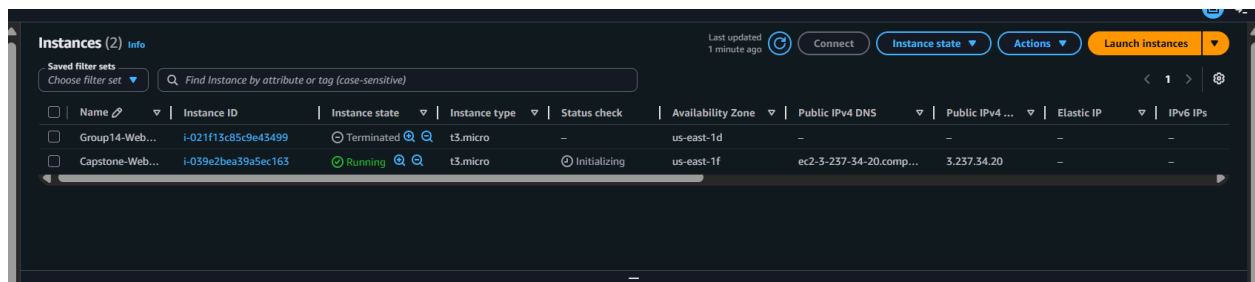
EC2 instance and S3 bucket

AWS Compute & Storage Infrastructure

- **Active EC2 Web Server:** This is our EC2 dashboard showing our compute layer. We have a running instance named Capstone-WebServer... utilizing the standard Free Tier t3.micro instance type. It is fully initialized in the us-east-1f availability zone with a public IPv4 address (3.237.34.20) assigned and ready to host your web server software.

CAPSTONE PROJECT PHASE 1

- **S3 Bucket Configuration:** This captures the initial creation screen for your object storage. You've chosen a standard **General purpose** bucket type set to use the standard regional namespace. The bucket is globally uniquely named capstone-static-assets-azubi-61035689714-us-east-1-an.
- **Static Asset Verification:** The final screenshot confirms your storage layer is working perfectly. You have successfully uploaded your first test static asset—an image named Azubi.png—into the root of the bucket using the **Standard** storage class, verifying that your upload permissions are working flawlessly.



CAPSTONE PROJECT PHASE 1

PHASE 2

Created a CloudFront distribution with ALB as the origin

The screenshot shows the 'Specify origin' step in the AWS CloudFront console. On the left, a progress bar indicates the steps: Step 1: Get started, Step 2: Specify origin (active), Step 3: Enable security, Step 4: Review and create. The main area is titled 'Specify origin' and contains two sections: 'Origin type' and 'Origin'. The 'Origin type' section has five radio buttons: 'Amazon S3' (selected), 'Elastic Load Balancer', 'API Gateway', 'Elemental MediaPackage', and 'VPC origin'. The 'Origin' section has two sub-sections: 'S3 origin' with a text input field containing 'example-bucket.s3.us-east-1.amazonaws.com' and a 'Browse S3' button, and 'Origin path - optional' with a text input field containing '/path'. At the bottom, there is a 'Settings' link.

The screenshot shows the 'student planner' distribution details in the AWS CloudFront console. At the top, a green banner states 'Successfully created new distribution.' Below this, the distribution name 'student planner' is shown with a 'Standard' plan tag and a 'View metrics' button. The 'Details' section includes: 'Distribution domain name' (dk24845v6mvo0.cloudfront.net), 'Billing' (Pay-as-you-go with a 'Switch to a plan' button), 'ARN' (arn:aws:cloudfront::610356897914:distribution/E171Y3DHLJ70BV), and 'Last modified' (Deploying). Below the details are tabs for 'General', 'Security', 'Origins', 'Behaviors', 'Error pages', 'Invalidations', 'Logging', and 'Tags'. The 'General' tab is active, showing 'Settings' with fields for 'Name' (student planner), 'Description' (-), 'Price class' (Use all edge locations (best performance)), and 'Supported HTTP versions' (HTTP/2, HTTP/1.1, HTTP/1.0). There are also sections for 'Alternate domain names' (with an 'Add domain' button), 'Standard logging' (Off), 'Cookie logging' (Off), 'Default root object' (-), and 'Cache tag header' (-). An 'Edit' button is located at the top right of the settings section.